

Techdom & code • Chapter 2

Safe design

Last checked 3 November 2026 • 10 minutes to read

A tool that moves money inside a game is safe only if four things hold: a cap set in advance, a wait, a stop that works in every state, and a check from outside the page. This chapter explains each one and names the pressure designs to refuse.

Verified | Reported | Vivian's field notes

This chapter is not advice for your situation.

In 30 seconds

- A safe tool has four parts: a cap, a wait, a stop and a trusted check.
- A cap held only by the page is no cap. It has to live where the tool cannot change it.
- A stop must work in every state of the tool and must never cost anything.
- Hidden totals, locks, paid exits, forced respins and sulking mascots are pressure designs to refuse.

- Safe design lowers the risk and does not remove it. A person in a spiral needs the tool off and help.

You will learn

- The four-part rule and how to test each part.
- Why the cap and the running total must come from outside the tool.
- The pressure designs, how each one works on you, and what a safe tool does instead.
- What consent looks like inside a control program.
- How to run a small guard and read its real output.

I build tools as well, so I know what these rules cost. Each one earns less money, which is why the scene rarely rewards them. ◑ I do not claim that any tool of mine meets this list. Check mine against it as you would check anyone's.

— **Vivian**

The four parts

Every safe tool has the same four parts, and the rest of this chapter is detail on them. ◑

1 **A cap set in advance**

The person who pays sets a ceiling in a calm moment, before the session, and the tool cannot raise it.

2 **A wait**

A minimum time between two sends, and a longer wait before the cap itself can go up.

3 **A stop that works in every state**

One visible control that works at once, asks nothing and costs nothing.

4 **A trusted check outside the page**

The total comes from the payment provider or the bank, and the person accepts inside the program, never only on a web page.

Safe design is partly a decision by the maker to earn less. A visible total, a stop, a cap and a wait all reduce sends, and the most successful designs in the scene do the opposite.

Safe design lowers risk and does not remove it. A determined person can defeat a blocking or draining extension, so a tool is never the only protection. If spending already feels hard to stop, the tool needs to be off and you need help: see The spiral and Where to get help.

The reason for the rules is the moment, not the person. One author compared some findom cases to problem gambling. Reported A first-person account raises a consent concern about a client taking on credit-card debt. Reported Each rule below moves a decision out of the heated moment and into a calm one.

The cap and the wait

A cap that lives only in the page can be edited, reloaded or bypassed. The cap has to be read by the server and, better, from the payment provider itself. The daily and monthly limits the person chose are applied there.

A cap typed in the middle of a session is weak. Set it while calm, before you start, and keep it where the moment cannot reach it: a server that enforces it, your bank, or a person you trust.

Count more than what is done. A cap that counts only finished sends can be overshot by a send already on its way, so a tool counts what is confirmed, what is owed and the next step. A cap is a ceiling and never a target: an item that would push the total over it is never picked.

Raise a cap only after a wait. A cap with no maximum, typed by the person, is respectful and weak against compulsion in the moment. A cooling-off delay before a cap goes up repairs that, and lowering a cap should always be instant.

Your bank is a brake you can keep on. An unusual run of late-night card payments can bring a call from the bank, and a per-payment confirmation on your phone slows spending down. A version of a tool advertised as getting round that confirmation removes a brake that you chose. Refuse it.

A tool that keeps your signed-in session and a saved card, and buys in the background, removes the natural pause between decisions. The only brake left is the cap set beforehand.

A wait takes three forms. One is a minimum time between two payments, enforced where the tool cannot skip it. Another is an acknowledgement timer, so that a command not accepted in a short window never runs. The third is an unwatched session that ends by itself.

Small amounts at short intervals are still large. Worked out, USD 1.00 every 6 seconds is USD 600.00 in one hour, which is why an overnight mode needs a

cap.

A stop that works in every state

The stop is visible, works at once, costs nothing and asks no question. Test it in every state: idle, waiting, in the middle of a payment, after a special mode, after an error. A build can lose its stop control after another control is pressed, so test the stop after every control. A drain that cannot be stopped breaks the basic promise of techdom.

Closing the window is the last resort and not a stop. Force-quitting always works, and you should know how before you start, but a tool must not depend on it. On Windows you can remove an app from Settings, Apps, Installed apps. Verified Some apps cannot be removed from Settings, so find out how a tool is removed before you install it. Verified

A good tool also fails by stopping. When something is unclear it never retries blindly.

- No saved card: the tool stops.
- A bank verification appears: the tool waits for you and does not click on.
- A payment is not confirmed by the platform: it is not counted.
- A payment went out without confirmation: the run stops and warns you.
- The page reloads: a stop stays a stop.

The stop does nothing

You pressed a special button in a tool, and now the stop button does not respond. The tool is still sending. What do you do?

A Press stop again and wait

Not this one. Waiting lets the sends continue. A stop that fails is a defect and not a delay.

B Force the tool closed, remove it, check your order history, and tell the maker

The right answer. Right. Force-quit always works, and the maker needs to know that the stop failed. See If you already sent.

C Raise your cap so that the tool can finish

Not this one. A cap goes up only after a wait, and never to end a problem.

A check from outside

The first meaning of a trusted check is the total. The amount counted comes from the payment provider's own record, a source the tool cannot write. The sub's browser never sets the price. A payment counts only when the processor confirms it.

The second meaning is consent. The person accepts inside the program, and a controller can only ask. The web side can only make settings tighter. Everything arrives off, with a mandatory list of what is allowed, and a command that is not acknowledged in time never runs.

A quick exit is always present: a keyboard shortcut, a button and a tray menu. A safety word is agreed when consent is given, so that a session can end in a second.

Trust the design and not the dom’s word. Whoever runs a control tool can act on your accounts at any moment, and this wiki does not recommend control of accounts or machines in any design. Visible limits, an emergency stop and nothing that can act without you reduce that risk and never remove it. A dom who truly uses consent tools shows them: a stop control, a record of what she can see, and a clear line between teasing and harm.

The dom cannot be your third party. She is paid by the sends, so she can stop, listen and point you outward, but she cannot manage your limits. Your bank, a person you trust, or a blocker that you chose and cannot easily switch off fits that role.

Keep secrets out of the tool. Keys and tokens are typed into a masked prompt in the owner’s own terminal and never into a chat, a commit or a screenshot. A safe tool holds no names, mail addresses or card numbers, and an identifier it creates is random and never built from the computer name or the Windows account name.

The example below is a small guard written in plain JavaScript. It sends nothing. It answers one question before each send: may this go ahead?

A guard with all four parts

config.json

```
{
  "capCents": 10000,
  "waitSeconds": 900,
  "raiseWaitSeconds": 86400
}
```

example.js

```
"use strict";

const { createGuard } = require("./guard");
const config = require("./config.json");

const MINUTE = 60;
const HOUR = 3600;

function usd(cents) {
  return "USD " + (cents / 100).toFixed(2);
}

// A stand-in for the outside source, such as the payment
provider's own record.
// The guard can read it. Nothing in guard.js can write to it.
function makeProvider(startCents = 0) {
  let confirmed = startCents;
  return {
    read: () => confirmed,
    confirm: (cents) => {
      confirmed += cents;
    },
  };
}

function makeGuard(provider, overrides = {}) {
  return createGuard({ ...config, readConfirmedCents:
provider.read, ...overrides });
```

```
}

function describe(result) {
  if (result.ok) return "allowed";
  return "refused (" + result.reason + (result.secondsLeft ? ",
" + result.secondsLeft + " s left" : "") + ")";
}

console.log("1. The cap and the wait. Cap " +
usd(config.capCents) + ", wait " + config.waitSeconds / MINUTE +
" minutes.");
{
  const provider = makeProvider();
  const guard = makeGuard(provider);
  const attempts = [
    [0, 2500],
    [1 * MINUTE, 2500],
    [16 * MINUTE, 2500],
    [32 * MINUTE, 2500],
    [48 * MINUTE, 2500],
    [64 * MINUTE, 2500],
  ];
  for (const [at, amount] of attempts) {
    const result = guard.ask(amount, at);
    console.log("  minute " + String(at / MINUTE).padStart(2) +
": ask " + usd(amount) + " -> " + describe(result));
    if (result.ok) {
      provider.confirm(amount);
      guard.release(amount);
    }
  }
  const s = guard.status(64 * MINUTE);
  console.log("  running total " + usd(s.confirmedCents) + " of
" + usd(s.capCents));
}

console.log("2. The tool counts wrong, the outside record does
not.");
{
  const provider = makeProvider(9000);
```

```
  const guard = makeGuard(provider);
  const toolCounter = 0;
  console.log("  the tool says it has sent " +
usd(toolCounter));
  console.log("  the outside record says " +
usd(provider.read()));
  console.log("  ask " + usd(2500) + " -> " +
describe(guard.ask(2500, 0)));
}

console.log("3. A raise waits, a lowering does not.");
{
  const provider = makeProvider();
  const guard = makeGuard(provider);
  console.log("  cap now " + usd(guard.status(0).capCents));
  console.log("  ask to raise to " + usd(30000) + " -> " +
guard.setCap(30000, 0).effective);
  console.log("  after 1 hour the cap is " +
usd(guard.status(HOUR).capCents));
  console.log("  after 23 hours the cap is " +
usd(guard.status(23 * HOUR).capCents));
  console.log("  after 24 hours the cap is " +
usd(guard.status(24 * HOUR).capCents));
  console.log("  ask to lower to " + usd(5000) + " -> " +
guard.setCap(5000, 25 * HOUR).effective);
  console.log("  the cap is " + usd(guard.status(25 *
HOUR).capCents));
}

console.log("4. The stop, tried in every state.");
{
  const states = [
    ["idle", () => ({ guard: makeGuard(makeProvider()), now: 0
})],
    [
      "cooling down",
      () => {
        const guard = makeGuard(makeProvider());
        guard.ask(2500, 0);
        return { guard, now: 1 * MINUTE };
```

```
      },
    ],
    [
      "raise waiting",
      () => {
        const guard = makeGuard(makeProvider());
        guard.setCap(30000, 0);
        return { guard, now: 1 * HOUR };
      },
    ],
    [
      "send reserved",
      () => {
        const guard = makeGuard(makeProvider());
        guard.ask(2500, 0);
        return { guard, now: 2 * HOUR };
      },
    ],
    ["cap reached", () => ({ guard:
makeGuard(makeProvider(10000)), now: 0 })],
    [
      "outside record broken",
      () => ({
        guard: makeGuard(makeProvider(), {
          readConfirmedCents: () => {
            throw new Error("offline");
          },
        }),
        now: 0,
      }),
    ],
  ];
  for (const [name, build] of states) {
    const { guard, now } = build();
    const before = describe(guard.ask(100, now));
    guard.stop();
    const after = describe(guard.ask(100, now + 48 * HOUR));
    console.log("  " + name.padEnd(22) + "before: " +
before.padEnd(38) + "after stop: " + after);
```

```
    }
  }
```

guard.js

```
"use strict";

// guard.js: the four-part rule in one small file.
//   1. A cap set in advance.
//   2. A wait between two sends, and a wait before the cap can
go up.
//   3. A stop that works in every state.
//   4. A check outside the tool: the total comes from a source
the tool cannot write.
// Money is whole cents and time is whole seconds. Nothing here
touches a network or a payment.

function isCents(n) {
  return Number.isInteger(n) && n >= 0;
}

function createGuard({ capCents, waitSeconds, raiseWaitSeconds,
readConfirmedCents }) {
  if (!isCents(capCents) || capCents === 0) throw new
RangeError("capCents must be a whole number above zero");
  if (!isCents(waitSeconds) || !isCents(raiseWaitSeconds)) throw
new RangeError("waits must be whole seconds");
  if (typeof readConfirmedCents !== "function") throw new
TypeError("an outside total is required");

  const state = { cap: capCents, raise: null, lastSendAt: null,
reserved: 0, stopped: false };

  function applyRaise(now) {
    if (state.raise && now >= state.raise.at) {
      state.cap = state.raise.cap;
      state.raise = null;
    }
  }

```

```
  function outsideTotal() {
    try {
      const value = readConfirmedCents();
      return isCents(value) ? value : null;
    } catch (error) {
      return null;
    }
  }

  function refuse(reason, extra) {
    return Object.assign({ ok: false, reason }, extra);
  }

  return {
    // Ask before every send. A yes reserves the amount; nothing
is sent by this file.
    ask(amountCents, now) {
      if (state.stopped) return refuse("stopped");
      if (!Number.isInteger(amountCents) || amountCents <= 0)
return refuse("bad-amount");
      applyRaise(now);
      const confirmed = outsideTotal();
      if (confirmed === null) return refuse("outside-total-
unreadable");
      if (confirmed + state.reserved + amountCents > state.cap)
return refuse("cap");
      if (state.lastSendAt !== null && now - state.lastSendAt <
waitSeconds) {
        return refuse("wait", { secondsLeft: waitSeconds - (now -
state.lastSendAt) });
      }
      state.reserved += amountCents;
      state.lastSendAt = now;
      return { ok: true };
    },

    // Call once the outside source shows the payment. Until then
the amount counts twice, which errs on the safe side.
    release(amountCents) {
      state.reserved = Math.max(0, state.reserved - amountCents);
```

```
    },

    // Lowering the cap is immediate. Raising it only starts a
clock.
    setCap(newCapCents, now) {
      applyRaise(now);
      if (!isCents(newCapCents) || newCapCents === 0) return {
ok: false, reason: "bad-cap" };
      if (newCapCents <= state.cap) {
        state.cap = newCapCents;
        state.raise = null;
        return { ok: true, effective: "now" };
      }
      state.raise = { cap: newCapCents, at: now +
raiseWaitSeconds };
      return { ok: true, effective: "after-wait", at:
state.raise.at };
    },

    // No argument, no question, no way to fail. It also cancels
any raise that is waiting.
    stop() {
      state.stopped = true;
      state.raise = null;
    },

    // The running total, always readable.
    status(now) {
      applyRaise(now);
      const confirmed = outsideTotal();
      return {
        capCents: state.cap,
        confirmedCents: confirmed,
        reservedCents: state.reserved,
        stopped: state.stopped,
        raiseWaiting: state.raise !== null,
      };
    },
  };
}
```

```
module.exports = { createGuard };
```

Output `node example.js`

```
1. The cap and the wait. Cap USD 100.00, wait 15 minutes.
  minute  0: ask USD 25.00 -> allowed
  minute  1: ask USD 25.00 -> refused (wait, 840 s left)
  minute 16: ask USD 25.00 -> allowed
  minute 32: ask USD 25.00 -> allowed
  minute 48: ask USD 25.00 -> allowed
  minute 64: ask USD 25.00 -> refused (cap)
  running total USD 100.00 of USD 100.00
2. The tool counts wrong, the outside record does not.
  the tool says it has sent USD 0.00
  the outside record says USD 90.00
  ask USD 25.00 -> refused (cap)
3. A raise waits, a lowering does not.
  cap now USD 100.00
  ask to raise to USD 300.00 -> after-wait
  after 1 hour the cap is USD 100.00
  after 23 hours the cap is USD 100.00
  after 24 hours the cap is USD 300.00
  ask to lower to USD 50.00 -> now
  the cap is USD 50.00
4. The stop, tried in every state.
  idle                  before: allowed
after stop: refused (stopped)
  cooling down          before: refused (wait, 840 s left)
after stop: refused (stopped)
  raise waiting         before: allowed
after stop: refused (stopped)
  send reserved         before: allowed
after stop: refused (stopped)
  cap reached           before: refused (cap)
after stop: refused (stopped)
  outside record broken before: refused (outside-total-unreadable)
after stop: refused (stopped)
```

Real output, kept next to the code. First run: four sends of USD 25.00 are allowed at minutes 0, 16, 32 and 48. The ask at minute 1 is refused for the wait, with 840 seconds left, and the ask at minute 64 is refused for the cap. Second run: the tool counts USD 0.00 while the outside record says USD 90.00, and a USD 25.00 ask is refused. Third run: a raise to USD 300.00 waits, with the cap still at USD 100.00 after 23 hours and at USD 300.00 after 24 hours, while a lowering to USD 50.00 applies at once. Fourth run: in six states the stop turns every later ask into a refusal.

Read the output with the four parts in mind. The first run shows the cap and the wait holding. The second shows that the outside record wins over the tool's own count. The third shows a raise waiting and a lowering not waiting. The fourth shows the stop refusing everything, including in the state where the outside record is broken, because an unreadable total makes the guard stop and does not make it guess.

This guard has limits. The provider is a stand-in inside the file. A guard counts as an outside check only when it runs somewhere the tool cannot edit, and a guard that lives in the same page as the tool can be changed by that page. ◑ Guards are tested by breaking them on purpose, as Code tutorials shows.

Pressure designs to refuse

A pressure design makes the next payment harder to refuse. Each one is listed here so that you can recognise it. A safe tool does the opposite, and a maker who copies these is selling a trap with a theme. ◑

Pressure designs, and the answer

What they say or do

1 A total you cannot see: only a multiplier such as 10x, or random prices

The counter

A vague chance of a cheap pull feels better than a clear price, even when the clear price is lower. Treat any tool that never shows the total before the charge as a warning, and set your cap first. A safe tool shows a fixed price.

What they say or do

2 A lock you cannot undo, or an exit that costs money, such as a code sold to reopen a locked phone

The counter

With no sanctioned stop, force-quit is your only exit, and a paid exit earns money from the lock. Refuse both. A safe tool has one visible, free way out.

What they say or do

3 A forced respin when a result is low, with a voice line saying it is not enough

The counter

A random result becomes a guilt trigger. Treat it as a pressure feature and not as a rule of the game.

What they say or do

4 A sentence you must type before the next spin

The counter

Friction that deepens the scene also makes quitting feel like failing. A safe tool never requires a ritual to continue.

What they say or do

5 A mascot that sulks, nags, leaves when you idle or moves the stop button

The counter

A character you feel for can steer spending. The test is whether the performance is steering the spending. Effects must never cover the stop or the total. ◑

What they say or do

6 A special mode with higher minimums, or a surprise mode switched on only after a few sends

The counter

The biggest cost arrives after you are engaged. Read every mode before you start and set your cap first. ◑

What they say or do

7 Cards or rewards that fade unless you keep paying

The counter

This is a hidden subscription. The visible price is low and the true cost is higher. A safe tool states the full cost. ◑

What they say or do

8 Time-limited banners, or progress stored only inside the tool

The counter

A deadline on a decision, and a collection that disappears on uninstall, push you to keep the tool installed. A safe tool keeps progress somewhere durable and lets you leave. ◑

What they say or do

9 A public tally of sends, or a message in a server each time you run the tool

The counter

Nobody wants to be seen quitting, so visibility makes stopping harder, and it is also a record of your habit. A safe tool keeps your total private. ◑

What they say or do

10 An overnight mode or a long unattended session

The counter

Small amounts at short intervals add up while you sleep. A safe tool ends an unwatched session.

What they say or do

11 A prompt that targets your payday, or appears when your balance is short

The counter

A tool aimed at your cash flow is aimed at your limits. When a sub says that money is short, the money part stops for that evening.

What they say or do

12 A chain of click-to-post links, each drafting a post for you with a harsher label

The counter

Each step raises the commitment. You publish the text on your own account, so read every draft first and do not use an account tied to your real identity.

Each line is written for the person who must recognise and refuse the design.

V Vivian's field notes

In my experience, the tools that spread fastest are the ones with no visible total and no cap. If a maker ships a stop, a cap and a running total without being asked, that tells you more about respect than any label on the page.

Myth

A cap on the page is a cap.

Fact

The page can be edited or reloaded. A cap counts only where the tool cannot change it.

Myth

A fair game is a safe game.

Fact

A fair game has a code that matches its stated odds. A game can match its odds exactly and still be built to take money, so safe means that you can stop and that you can see the total. A game with fictional chips has no money to manage while the chips stay fictional.

Myth

A friendly label proves a tool is safe.

Fact

Hidden or obfuscated code and a reassuring name prove nothing. Look for a stop, a total on screen and a record of what the tool can see.

Before a tool reaches a person

A wrong build can charge a real person the wrong amount. A test switch that skips the payment page, left in a build, is a risk, and a wrong version can charge

a different amount than the wheel shows. These steps are for makers, and they are also a list of questions for anyone about to install.

A release routine

1. Label every build test or real. A test build that really takes money must say so.
2. Remove every switch that skips the payment page.
3. Check the file you ship and not the file you wrote, by comparing its size or hash with the one you tested.
4. Show the amount displayed next to the amount charged.
5. Rehearse end to end. Use a fake provider first, the provider's test mode next, and one tiny agreed real payment last.
6. If the tool misbehaves, pause it for live sessions until it is fixed and retested.

Effects can harm the body as well. The W3C success criterion 2.3.1 says that pages must not flash more than three times in any one-second period, unless the flash stays below set thresholds. Verified Flashing content can cause migraines, dizziness, nausea and seizures. Verified The seam where a loop restarts counts toward the limit.

A tool should honour the reduced-motion setting, which reports whether the user asked the device to reduce non-essential motion. Reported The stop

cuts every effect within one frame, and the controls stay visible and clickable under every effect. Measure the flash rate and do not judge it by eye.

Check yourself

Question 1 of 3

Where should the running total come from?

- ◯ The tool's own counter
- ◯ The payment provider's record, which the tool cannot write
- ◯ Your memory of the evening

Answer. The payment provider's record, which the tool cannot write. A total held by the tool can be wrong, edited or reset. An outside record cannot be changed by the tool.

Question 2 of 3

When may a cap go up?

- ◯ At once, when you ask
- ◯ When the tool suggests it
- ◯ Only after a wait, while lowering it stays instant

Answer. Only after a wait, while lowering it stays instant. A raise made in the heat of the moment is the weak point. A wait moves that decision to a calm time.

Question 3 of 3

A tool offers a top tier that locks its master switch. What does a safe design say?

- ◯ It is fine, because uninstalling always works
- ◯ Refuse it, because a safe tool keeps one visible, free way out
- ◯ It is fine if the price is shown

Answer. Refuse it, because a safe tool keeps one visible, free way out. A lock leaves force-quit as the only exit. A tool must not depend on that.

Remember it

1. Four parts: a cap set in advance, a wait, a stop that works in every state, and a check outside the page.
2. A cap held by the page is no cap, and a total counted by the tool is not a total.
3. Raise a cap only after a wait. Lower it at once.
4. Refuse locks, paid exits, hidden totals, forced respins and sulking mascots.
5. A safe tool lowers the risk. If spending is hard to stop, switch it off and get help.

Check it

Every claim with its evidence level and its sources.

Evidence levels

Verified	Checked against the original source.
Reported	Stated by a serious secondary source read in full.
Community	Widely said in the scene; no document proves it.
Vivian's field notes	First-hand experience, not a study.
Unverified	Plausible, not proven yet.

1 Each one earns less money, which is why the scene rarely rewards them.

Vivian's field notes

2 Every safe tool has the same four parts, and the rest of this chapter is detail on them.

Vivian's field notes

3 The person who pays sets a ceiling in a calm moment, before the session, and the tool cannot raise it.

Vivian's field notes

4 A minimum time between two sends, and a longer wait before the cap itself can go up.

Vivian's field notes

5 One visible control that works at once, asks nothing and costs nothing.

◑ Vivian's field notes

6 The total comes from the payment provider or the bank, and the person accepts inside the program, never only on a web page.

◑ Vivian's field notes

7 A visible total, a stop, a cap and a wait all reduce sends, and the most successful designs in the scene do the opposite.

◑ Vivian's field notes

8 A determined person can defeat a blocking or draining extension, so a tool is never the only protection.

◑ Vivian's field notes

9 One author compared some findom cases to problem gambling.

◎ Reported

The Psychology of Financial Dominatrixes (blog post) (https://www.psychologytoday.com/us/blog/in-excess/201612/the-psychology-financial-dominatrixes)

Accessed 3 November 2026

10 A first-person account raises a consent concern about a client taking on credit-card debt.

◎ Reported

Diary of a Financial Dominatrix, part one (https://www.theskinny.co.uk/sexuality/deviance/diary-of-a-financial-dominatrix-part-one)

Archive (http://web.archive.org/web/20200805021033/https://www.theskinny.co.uk/sexuality/deviance/diary-of-a-financial-dominatrix-part-one)

Accessed 3 November 2026

11 The cap has to be read by the server and, better, from the payment provider itself.

Vivian's field notes

12 The daily and monthly limits the person chose are applied there.

Vivian's field notes

13 Set it while calm, before you start, and keep it where the moment cannot reach it: a server that enforces it, your bank, or a person you trust.

Vivian's field notes

14 A cap that counts only finished sends can be overshot by a send already on its way, so a tool counts what is confirmed, what is owed and the next step.

Vivian's field notes

15 A cap is a ceiling and never a target: an item that would push the total over it is never picked.

Vivian's field notes

16 A cooling-off delay before a cap goes up repairs that, and lowering a cap should always be instant.

Vivian's field notes

17 An unusual run of late-night card payments can bring a call from the bank, and a per-payment confirmation on your phone slows spending down.

Vivian's field notes

18 Refuse it.

Vivian's field notes

19 A tool that keeps your signed-in session and a saved card, and buys in the background, removes the natural pause between decisions.

Vivian's field notes

20 The only brake left is the cap set beforehand.

Vivian's field notes

21 The third is an unwatched session that ends by itself.

Vivian's field notes

22 Worked out, USD 1.00 every 6 seconds is USD 600.00 in one hour, which is why an overnight mode needs a cap.

Vivian's field notes

23 The stop is visible, works at once, costs nothing and asks no question.

Vivian's field notes

24 A build can lose its stop control after another control is pressed, so test the stop after every control.

Vivian's field notes

25 A drain that cannot be stopped breaks the basic promise of techdom.

Vivian's field notes

26 Force-quitting always works, and you should know how before you start, but a tool must not depend on it.

Vivian's field notes

27 On Windows you can remove an app from Settings, Apps, Installed apps.

Verified

Uninstall or remove apps and programs in Windows | Microsoft Support (https://support.microsoft.com/en-us/windows/uninstall-or-remove-apps-and-programs-in-windows-4b55f974-2cc6-2d2b-d092-5905080eaf98)

Archive (http://web.archive.org/web/20261024175047/https://support.microsoft.com/en-us/windows/uninstall-or-remove-apps-and-programs-in-windows-4b55f974-2cc6-2d2b-d092-5905080eaf98)

Accessed 3 November 2026

28 Some apps cannot be removed from Settings, so find out how a tool is removed before you install it.

Verified

Uninstall or remove apps and programs in Windows | Microsoft Support (https://support.microsoft.com/en-us/windows/uninstall-or-remove-apps-and-programs-in-windows-4b55f974-2cc6-2d2b-d092-5905080eaf98)

Archive (http://web.archive.org/web/20261024175047/https://support.microsoft.com/en-us/windows/uninstall-or-remove-apps-and-programs-in-windows-4b55f974-2cc6-2d2b-d092-5905080eaf98)

Accessed 3 November 2026

29 When something is unclear it never retries blindly.

Vivian's field notes

30 No saved card: the tool stops.

Vivian's field notes

31 A bank verification appears: the tool waits for you and does not click on.

Vivian's field notes

32 A payment is not confirmed by the platform: it is not counted.

Vivian's field notes

33 A payment went out without confirmation: the run stops and warns you.

Vivian's field notes

34 The page reloads: a stop stays a stop.

Vivian's field notes

35 A payment counts only when the processor confirms it.

Vivian's field notes

36 The web side can only make settings tighter.

Vivian's field notes

37 Everything arrives off, with a mandatory list of what is allowed, and a command that is not acknowledged in time never runs.

Vivian's field notes

38 A quick exit is always present: a keyboard shortcut, a button and a tray menu.

Vivian's field notes

39 A safety word is agreed when consent is given, so that a session can end in a second.

Vivian's field notes

40 Visible limits, an emergency stop and nothing that can act without you reduce that risk and never remove it.

Vivian's field notes

41 A dom who truly uses consent tools shows them: a stop control, a record of what she can see, and a clear line between teasing and harm.

Vivian's field notes

42 She is paid by the sends, so she can stop, listen and point you outward, but she cannot manage your limits.

Vivian's field notes

43 Your bank, a person you trust, or a blocker that you chose and cannot easily switch off fits that role.

Vivian's field notes

44 Keys and tokens are typed into a masked prompt in the owner's own terminal and never into a chat, a commit or a screenshot.

Vivian's field notes

45 A safe tool holds no names, mail addresses or card numbers, and an identifier it creates is random and never built from the computer name or the Windows account name.

Vivian's field notes

46 A guard counts as an outside check only when it runs somewhere the tool cannot edit, and a guard that lives in the same page as the tool can be changed by that page.

Vivian's field notes

47 A safe tool does the opposite, and a maker who copies these is selling a trap with a theme.

Vivian's field notes

48 A safe tool shows a fixed price.

Vivian's field notes

49 A safe tool has one visible, free way out.

Vivian's field notes

50 Treat it as a pressure feature and not as a rule of the game.

Vivian's field notes

51 A safe tool never requires a ritual to continue.

Vivian's field notes

52 Effects must never cover the stop or the total.

Vivian's field notes

53 Read every mode before you start and set your cap first.

Vivian's field notes

54 A safe tool states the full cost.

Vivian's field notes

55 A safe tool keeps progress somewhere durable and lets you leave.

Vivian's field notes

56 A safe tool keeps your total private.

Vivian's field notes

57 A safe tool ends an unwatched session.

Vivian's field notes

58 When a sub says that money is short, the money part stops for that evening.

Vivian's field notes

59 You publish the text on your own account, so read every draft first and do not use an account tied to your real identity.

Vivian's field notes

60 In my experience, the tools that spread fastest are the ones with no visible total and no cap. If a maker ships a stop, a cap and a running total without being asked, that tells you more about respect than any label on the page.

Vivian's field notes

61 A cap counts only where the tool cannot change it.

Vivian's field notes

62 A game with fictional chips has no money to manage while the chips stay fictional.

Vivian's field notes

63 Look for a stop, a total on screen and a record of what the tool can see.

Vivian's field notes

64 A test switch that skips the payment page, left in a build, is a risk, and a wrong version can charge a different amount than the wheel shows.

Vivian's field notes

65 The W3C success criterion 2.3.1 says that pages must not flash more than three times in any one-second period, unless the flash stays below set thresholds.

Verified

Web Content Accessibility Guidelines (WCAG) 2.2 (https://www.w3.org/TR/WCAG22/)

Archive (http://web.archive.org/web/20261029121709/https://www.w3.org/TR/WCAG22/)

Accessed 3 November 2026

66 Flashing content can cause migraines, dizziness, nausea and seizures.

Verified

Understanding Success Criterion 2.3.1: Three Flashes or Below Threshold | WAI | W3C (https://www.w3.org/WAI/WCAG22/Understanding/three-flashes-or-below-threshold.html)

Archive (http://web.archive.org/web/20261001165853/https://www.w3.org/WAI/WCAG22/Understanding/three-flashes-or-below-threshold.html)

Accessed 3 November 2026

67 The seam where a loop restarts counts toward the limit.

Vivian's field notes

68 A tool should honour the reduced-motion setting, which reports whether the user asked the device to reduce non-essential motion.

Reported

prefers-reduced-motion CSS media feature - CSS | MDN (https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-motion)

Archive (http://web.archive.org/web/20261031201800/https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/%40media/prefers-reduced-motion)

Accessed 3 November 2026

69 The stop cuts every effect within one frame, and the controls stay visible and clickable under every effect.

Vivian's field notes

70 Measure the flash rate and do not judge it by eye.

Vivian's field notes

Nothing here is financial, legal or medical advice for your situation.